# 강의 계획

v하드웨어 설계 방법

ϖ하드웨어 기술 언어프로그램설치와 사용

ϖ조합논리회로 (기본 게이트/전가산기/디코더)

ϖ조합논리회로 (인코더/멀티플렉서/크기 비교기)

ϖ조합논리회로 (가산/감산기)

ϖ순차논리회로 (상태도 구현/ 레지스터 표현)

ϖ순차논리회로 (카운터/순차 검출기)

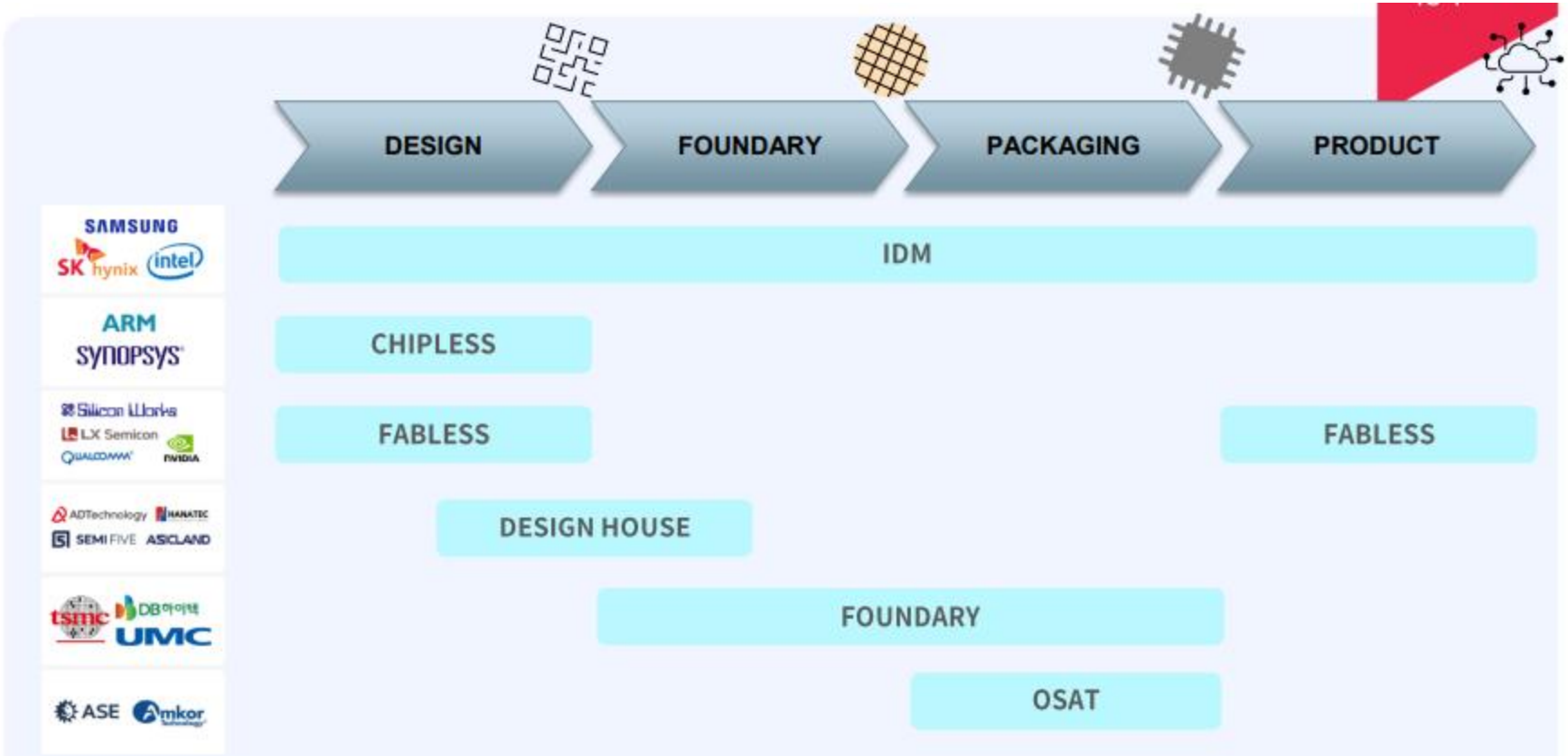
ϖ기타논리회로 (동기식 출력회로/스텝 클럭)

ϖ디지털 시계 설계 I

ϖ디지털 시계 설계 II

ϖ기타 응용 회로

# 반도체 생산과정 및 파트 별 주요 업체 현황



## 1. 종합 반도체 기업 (IDM, Integrated Device Manufacturer)

종합 반도체 기업(IDM, Integrated device manufacturer)은 모든 반도체 생산 공정을 종합적으로 갖춘 기업을 뜻한다.

한 회사가 웨이퍼 생산 설비인 **팹(fab)**을 갖추고 있고, 반도체 설계, 웨이퍼 가공, 패키징, 테스트 로 이어지는 반도체를 만들기 위한 일련의 과정을 모두 수행한다.

팹(fab)이란? ‘fabrication facility’의 약자로 웨이퍼를 생산하는 설비를 의미한다. 팹을 건설하려면 수 조원 이상의 대규모 자본이 필요하기 때문에 대기업이 주도하고 있다.

## 2.IP(intellectual property) 기업_chipless

IP기업은 팹리스처럼 반도체 ‘설계’를 전문으로 하는 회사이다. 하지만 팹리스와는 수익 모델이 조금 다르다.

IP기업은 셀 라이브러리라고 하는 특정 설계 블록을 팹리스나 IDM, 파운드리 등에 제공하고 IP 사용에 따른 라이선스료, 로열티를 받는다. 그래서 지적 재산권을 뜻하는 IP(intellectual property) 용어를 쓰는 것이다.

정리하면 팹리스는 설계 후 외주를 통해 자사 제품을 생산한다면, IP기업은 설계 라이선스를 판매할 뿐 자신의 브랜드로 제품을 생산하지 않는다.

## 3. 팹리스 (Fabless)

팹리스 는 반도체 설계를 전문적으로 하는 기업이다. 설계를 제외한 웨이퍼 생산, 패키징, 테스트 등은 모두 외주로 진행되며, 외주를 통해 생산이 완료된 칩의 소유권이나 영업권은 팹리스에 있어 자사 브랜드로 판매한다.팹리스 는 대규모 자본이 드는 공장을 갖추지 않고 설계에 주력할 수 있는 비즈니스 모델이다. 반도체를 만드는 생산시설 없이 뛰어난 아이디어와 우수한 칩 설계 기술을 바탕으로 반도체 칩 개발에 집중한다. 따라서 다품종 소량 생산 형태로 기술적인 다양성을 갖는 시스템 반도체가 주로 팹리스 의 형태를 가진다.

## 4. 디자인하우스(Design House)

좀 더 쉬운 이해를 위해 반도체 공정을 빵 만들기에 비유해보면, 빵 레시피를 만드는 사람을 팹리스(설계), 빵을 직접 구워내는 제빵사는 파운드리(생산)라고 볼 수 있다. 그리고 이제 빵을 구워야 하는데, 이때 밀가루를 최적의 비율로 믹스해서 제공받으면 훨씬 수월하다. 이처럼 생산이 용이하도록 중간 가교 역할을 하는 것이 바로 '디자인하우스'이다.

## 5. 파운드리

파운드리는 생산 공정을 전담하는 기업으로 자체 제품이 아닌 수탁생산을 주로 하고 있다. 즉 반도체 생산 설비를 갖추고 있지만 직접 설계하여 제품을 만드는 것이 아니라 고객으로부터 위탁받은 제품을 대신 생산해 이익을 얻는 것이다. 파운드리 기업은 자체적으로 IP를 설계하기도 하며, IP회사들과 제휴를 맺어 고객들이 필요로 할 시 좋은 IP를 제공하기도 한다.

생산 전문 파운드리의 주 고객은 바로 팹리스, 시스템 반도체 회사이다. 반도체 생산을 위해서는 수조원대의 막대한 시설 투자비용이 들고 고도의 생산기술이 필요해 반도체를 개발하는 모든 회사들이 반도체를 생산 하기는 어렵기 때문이다.

## 6. OSAT(Outsourced Semiconductor Assembly And Test)

OSAT(Outsourced Semiconductor Assembly And Test)는 반도체 패키징 및 테스트 수탁기업으로 어셈블리 기업,패키징 기업이라고도 불린다. 폭넓은 의미에서 반도체 후공정 업계를 뜻하는데, 반도체 후공정은 크게 패키징, 테스트로 볼 수 있다.반도체 팹 공정을 통해 만들어진 웨이퍼에 있는 수백 개의 칩이 있다. 그러나 이 상태의 칩은 외부와 전기신호를 주고받을 수 없으며, 외부 충격에 의해 손상되기도 쉽다.

이 칩을 낱개로 하나하나 잘라내어 기판이나 전자기기에 장착되기 위해 포장하는 작업을 패키징이라 한다.패키징 기업은 이런 반도체 칩 포장을 전문으로 하는 곳이다.이렇게 팹 공정과 패키징 공정을 마친 반도체는 완벽한 품질과 신뢰성을 위해 철저한 검사를 거친다. 고가의 전자 장비의 핵심 부품인 만큼 엄격한 신뢰성이 요구되기 때문이다.

IDM이나 파운드리 기업은 테스트 전문 기업에 업무를 위탁함으로써 효율을 높이고 전문성을 더하기도 한다.

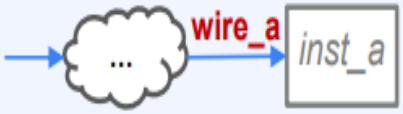
# data type

- Value Set
  - **0 : logic 0, false**
  - **1 : logic 1, true**
  - x : unknown
  - z : high-impedance, open connection

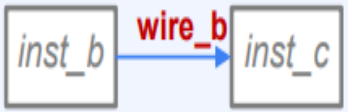
- Format
  - Binary : 'b
  - Decimal : ('d)
  - Hexa : 'h
  - *Ex) a = 'b1010; a = 'd10; a = 10; a = 'hA;*
  - *Ex) a = 4'b1010; a = 4'd10; a = 10; a = 4'hA;*
  - *cf) $display("binary %b, decimal %d, hexa %h",bin_a,dec_b,hex_c);*
  - *cf) a = '0; a = '1; a = 'x; a = 'z;*

# data type

- Nets
  - physical connection between entities
  - Do not store values
    This data type is always affected by driving circuit
  - Normally used in combinational logic
  - **wire**, tri, wand, wor, supply0, supply1, .....
  - *assign wire_a = ... ;*
    *module_a inst_a (wire_a)*




  - *module_b inst_b (wire_b);*
    *module_c inst_c (wire_b);*




- Variables
  - used in procedural blocks
  - Store values
  - Normally used in not only sequential logic but also combinational logic
  - **reg**, int, real, time
  - *initial begin reg_a = ...; end*
  - *always @(*) begin reg_b = ...; end*
  - *reg_a, reg_b keeps its value before next line*
  - *always @(posedge clk) begin reg_c <= ...; end*

# data type

- Vectors
  - nets, variables are usually 1 bit type
  - We can define bit width "N" like

    *wire [N-1:0] wire_a;*
    *reg [N-1:0] reg_a;*

- Array
  - Data type can have its dimensions
  - *[N-1:0]* after data type keyword means bit width=N (vector)
  - while *[M-1:0]* after variable name means its depth=M (scalar)
  - wire *[N-1:0] array_a [M-1:0];*
    *reg [N-1:0] array_b [M-1:0];*

```
wire        wire_1bit;
wire [7:0]  wire_8bit;

reg         reg_1bit;
reg  [7:0]  reg_8bit;

assign      wire_1bit = 1'b0;
assign      wire_8bit = 8'hFF;

always @* begin
    reg_1bit = 1'b1;
    reg_8bit = 8'hF0;
end

wire [7:0]  wire_8bit_2 = 'hF;
```

```
reg [3:0]   vector_a;               //4-bit width reg vector
reg         scalar_a [3:0];         //scalar reg array of depth 4, each 1-bit

reg [3:0]   array_1D [7:0];         //4-bit reg vector with depth 8, 1D array
reg [3:0]   array_2D [1:0][7:0];    //2D array with 2 rows, 8 columns, each 4-bit wide

always @* begin
    vector_a        = 'h0;

    //scalar_a      = 'h0;      //illegal
    scalar_a[0]     = 'h0;
    scalar_a[1]     = 'h0;
    //...

    array_1D[0]     = 'h0;
    array_2D[0][0]  = 'h0;
end
```

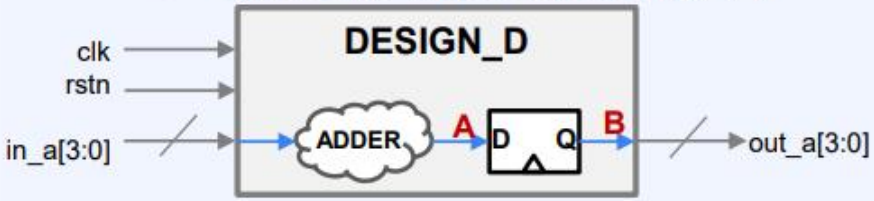
# test bench program

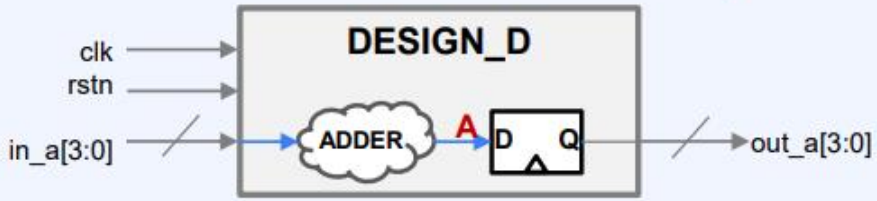
- Design module who get the output of AND/OR/XOR for 2-inputs
  - Design AND/OR/XOR module each, and instantiate in a module
  - refer to following as testbench code

```
module testbench;    //part2 ch3 clip4
    reg     in_a, in_b;
    wire    out_and, out_or, out_xor;

    initial begin
            {in_a,in_b} = 'b00;
        #10 {in_a,in_b} = 'b01;
        #10 {in_a,in_b} = 'b10;
        #10 {in_a,in_b} = 'b11;
        #10 $finish;
    end

    initial begin
        $dumpfile("test.vcd");
        $dumpvars(0,testbench);
    end

    design_gate    u_design_gate(
        .in_a       (in_a),
        .in_b       (in_b),
        .out_and    (out_and),
        .out_or     (out_or ),
        .out_xor    (out_xor)
    );
endmodule
```

# Module Design

- Module design with Data type
  - out is Flip-Flop's output with +1



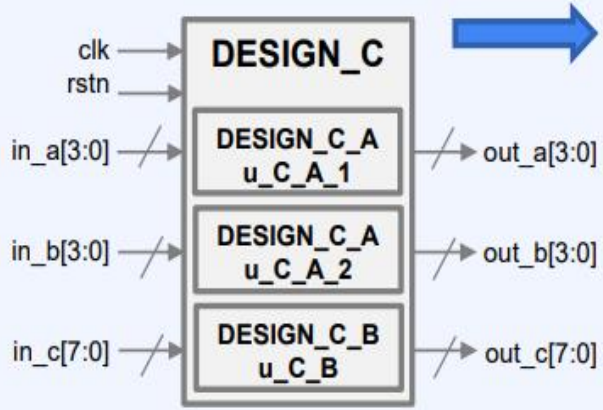

```
module DESIGN_D(
    input       clk,
    input       rstn,
    input [3:0] in,
    output[3:0] out
);
    wire  [3:0] a;
    reg   [3:0] b;

    assign      a = in + 1;

    always @(posedge clk, negedge rstn) begin
        if  (!rstn) b <= 0;
        else        b <= a;
    end

    assign      out = b;

endmodule
```

```
module DESIGN_D(
    input       clk,
    input       rstn,
    input [3:0] in,
    output[3:0] out
);
    wire  [3:0] a;
    reg   [3:0] out;

    assign      a = in + 1;

    always @(posedge clk, negedge rstn)
    if  (!rstn) out <= 0;
    else        out <= a;

endmodule
```

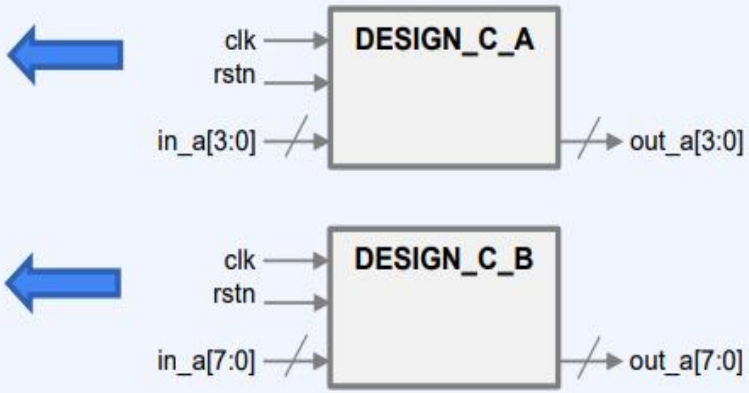
```
module DESIGN_D(
    input                clk,
    input                rstn,
    input        [3:0]   in,
    output reg   [3:0]   out
);
    wire         [3:0]   a;

    assign       a = in + 1;

    always @(posedge clk, negedg
    if  (!rstn) out <= 0;
    else        out <= a;

endmodule
```

# Instantiation

- Instantiation
  - Call sub-modules below module
  - Hierarchy between modules



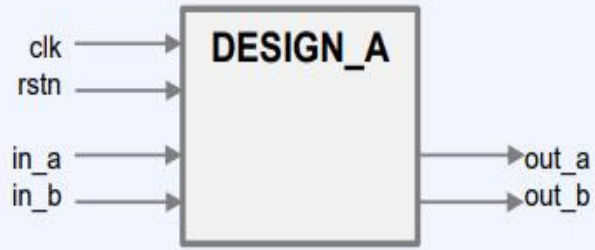

```
module DESIGN_C (
    input       clk,
    input       rstn,

    input [3:0] in_a,
    input [3:0] in_b,
    input [7:0] in_c,

    output[3:0] out_a,
    output[3:0] out_b,
    output[7:0] out_c
);
    //Instantiation for DESIGN_C
    DESIGN_C_A u_C_A_1(clk, rstn, in_a, out_a);

    DESIGN_C_A u_C_A_2(
        .clk    (clk    ),
        .rstn   (rstn   ),
        .in     (in_b   ),
        .out    (out_b  )
    );

    DESIGN_C_B u_C_B  (
        .clk    (clk    ),
        .rstn   (rstn   ),
        .in     (in_c   ),
        .out    (out_c  )
    );

    //Code for DESIGN_C
    //....
    //....

endmodule
```




```
module DESIGN_C_A (
    input       clk,
    input       rstn,
    input [3:0] in,
    output[3:0] out
);
    //Code for DESIGN_C_A

endmodule

module DESIGN_C_B (
    input       clk,
    input       rstn,
    input [7:0] in,
    output[7:0] out
);
    //Code for DESIGN_C_B

endmodule
```

# Module & Port

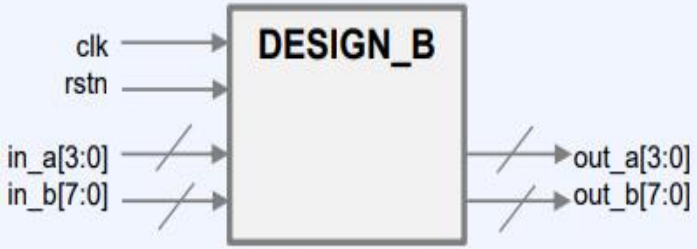
- Module
  - Design Unit in general
  - Usually, one module in a file with same name




```
module DESIGN_A (
    clk, rstn,
    in_a, in_b,
    out_a, out_b
);
    input   clk;
    input   rstn;
    input   in_a;
    input   in_b;
    output  out_a;
    output  out_b;

    //Code for DESIGN_A
    //....
    //....

endmodule
```

```
module DESIGN_A (
    input   clk,
    input   rstn,
    input   in_a,
    input   in_b,
    output  out_a,
    output  out_b
);

    //Code for DESIGN_A
    //....
    //....

endmodule
```

- Ports
  - input & output & (inout) ,
  - interface between modules
  - We can express bit width as data type




```
module DESIGN_B (
    input       clk,
    input       rstn,
    input [3:0] in_a,
    input [7:0] in_b,
    output[3:0] out_a,
    output[7:0] out_b
);

    //Code for DESIGN_B
    //....
    //....

endmodule
```

# PARAMETER

- What is parameter
  - Bit width for vector or array can be various depending on situation
  - for same function with different bit width, it's too inefficient to make lots of modules for each bit width
  - We can use parameters for variant number. We don't need to fix bit width which is variant
  - parameterized design is very recommended in field because it is very good to reuse

```
module PARAMETER_EX1(
    input       clk,
    input       rstn,
    input [3:0] in,
    output[3:0] out
);
    parameter   N=4;

    wire  [N-1:0]   a;
    reg   [N-1:0]   out;

    assign      a = in + 1;

    always @(posedge clk, negedge rstn)
    if  (!rstn) out <= 0;
    else        out <= a;

endmodule
```

```
module PARAMETER_EX2 #(
    parameter       N=4 //,
)(
    input           clk,
    input           rstn,
    input [N-1:0]   in,
    output[N-1:0]   out
);

    wire  [N-1:0]   a;
    reg   [N-1:0]   out;

    assign      a = in + 1;

    always @(posedge clk, negedge rstn)
    if  (!rstn) out <= 0;
    else        out <= a;

endmodule
```

# PARAMETER

- parameter override
  - Several bit width should be used at multiple instance in hierarchy
  - We can control parameter's value in upper hierarchy
  - defparam, parameter argument are the way
- localparam
  - This acts like parameter in a module but can't be overridden by upper hierarchy

```
module PARAMETER_EX3 #(
    parameter       N=4,
    parameter       M=8
)(
    input           clk,
    input           rstn,

    input [N-1:0]   in_1,
    input [M-1:0]   in_2,
    output[N-1:0]   out_1,
    output[M-1:0]   out_2,
);
    defparam u_EX3_1.N = N; //parameter 4
    PARAMETER_EX3_1 u_EX3_1 (in_a, out_a);

    PARAMETER_EX3_1 #(.N(M)) u_EX3_2 (in_a, out_a); //parameter 8
endmodule
```

```
module PARAMETER_EX3_1 #(
    parameter   N=8
)(
    input [N-1:0]   in,
    output[N-1:0]   out
);
    localparam  K=32;
    //Code for PARAMETER_EX3_1

endmodule
```

# define

- Usage
  - This role is similar with language C
  - Sometimes, several lines should be compiled or not depending on situation
    define is frequently used with ifdef/else
    for covering this case
  - *`define, `ifdef, `ifndef, `else* in Verilog
  - Do not use too much because it can make worse readability

```
`define SEQ_LOGIC
module DEFINE_EX #(
    parameter   N=8
)(
`ifdef SEQ_LOGIC
    input           clk,
    input           rstn,
`endif
    input [N-1:0]   in,
    output[N-1:0]   out
);

`ifndef SEQ_LOGIC
    assign  out = in;
`else
    reg [N-1:0] out;

    always @(posedge clk, negedge rstn)
    if  (!rstn) out <= 0;
    else        out <= in;
`endif

endmodule
```

# Operator

- Bit-part
  - wire [3:0] a;
  - wire[1:0] b, c;
  - b=a[3:2];
  - c=a[1:0];

- Concatenation
  - a = {b, c};
  - {b, c} = a;

  - Replication
    a = {2{b}, c};

  - Bit width of each variables are very important

- Conditional
  - a = b ? c : d;

# Operator

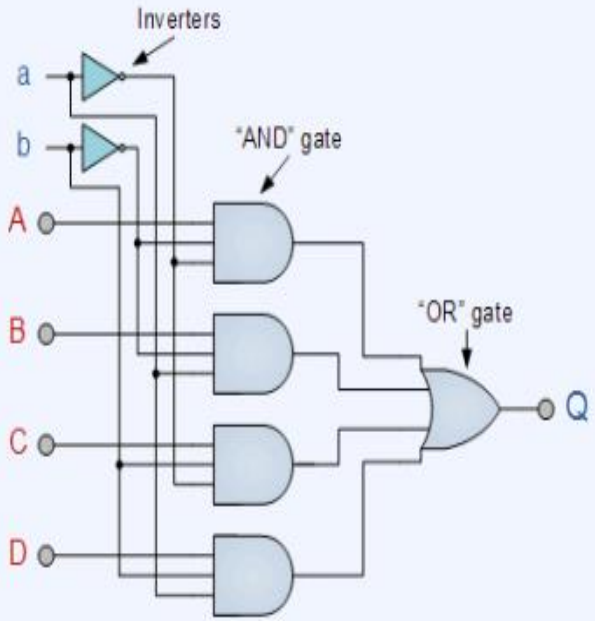
- Arithmetic
  - a+b; a-b; a*b; a/b; a%b; a**b;
- Relational
  - a<b; a>b; a<=b; a>=b;
- Equality
  - a===b; a!==b; a==b; a!=b;
- Logical
  - a && b; a || b; !a;
- Bitwise
  - ~a; a & b; a | b; a ^ b;
- Unary
  - &a; |a; ^a;
- Shift
  - a >> 3; a << 3; a >>> 3; a <<< 3;

# Modeling

- 3 types of Modeling
  - **Gate-level (Not much usage in real coding)**
  - **Dataflow**
  - **Behavioral**

# Gate Level Modeling

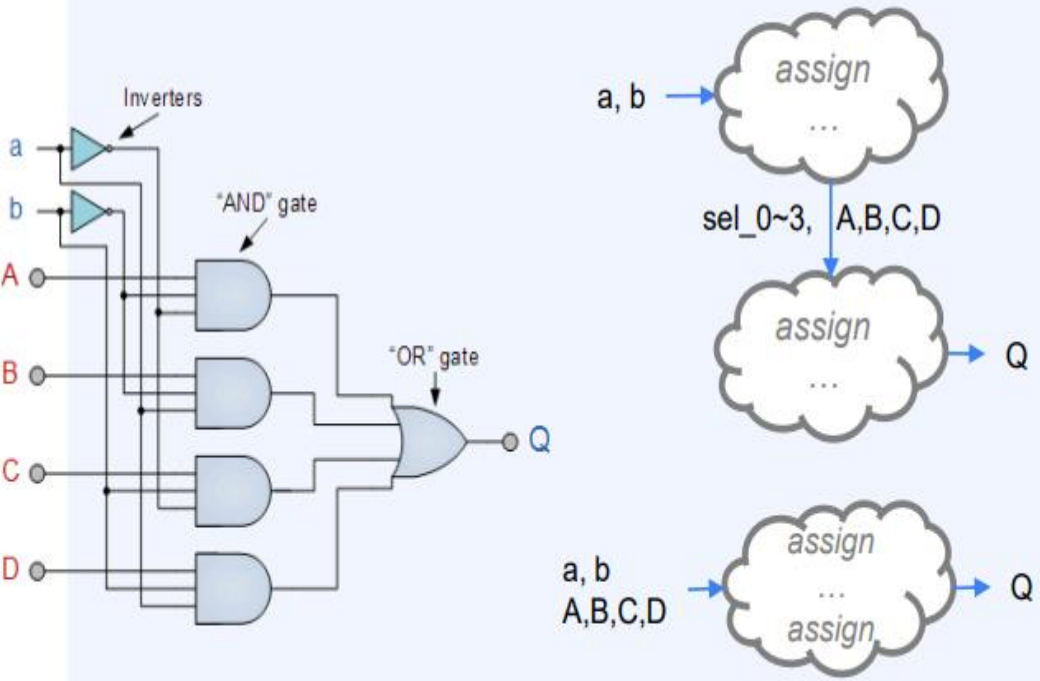
- Gate Level Modeling
  - Verilog can provide gate primitive cell which can be instantiated
  - Describe each gate with each instance including connection




```
module mux_gatelevel(
    input   a, b,
    input   A, B, C, D,
    output  Q
);
    wire    a_not, b_not;
    wire    A_pick, B_pick, C_pick, D_pick;

    not(a_not , a);
    not(b_not , b);

    and(A_pick, a_not, b_not, A);
    and(B_pick, a    , b_not, B);
    and(C_pick, a_not, b    , C);
    and(D_pick, a    , b    , D);

    or (Q, A_pick, B_pick, C_pick, D_pick);

endmodule
```

# Data Flow Modeling

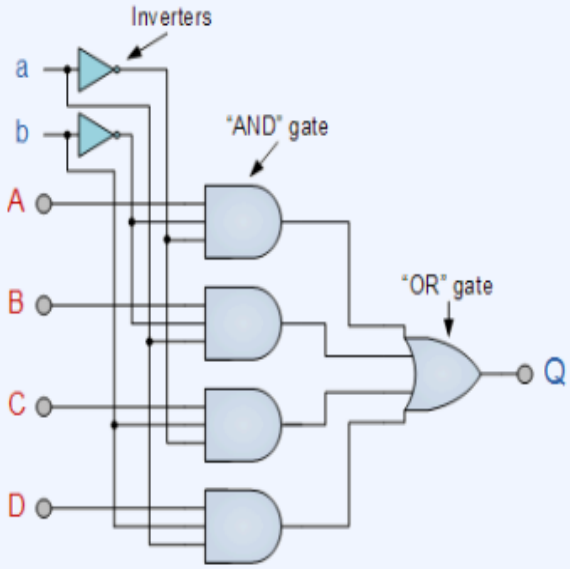
- Data Flow Modeling
  - assign statement is used with operators which is more effective to describe than gates
  - Between assign statements, there is no sequence but dependency
  - We can know how data flows in this modeling (which variables follow which operations)




```
module mux_dataflow(
    input   a, b,
    input   A, B, C, D,
    output  Q
);
    wire    sel_0, sel_1, sel_2, sel_3;

    assign  sel_0 = ~a & ~b;
    assign  sel_1 =  a & ~b;
    assign  sel_2 = ~a &  b;
    assign  sel_3 =  a &  b;

    //assign    Q = (sel_0 & A) | (sel_1 & B) | (sel_2 & C) | (sel_3 & D);

    assign  Q = sel_0 ? A :
                sel_1 ? B :
                sel_2 ? C :
                sel_3 ? D : 0;
    /*
    assign  Q = (~a & ~b) ? A :
                ( a & ~b) ? B :
                (~a &  b) ? C :
                ( a &  b) ? D : 0;
    */
endmodule
```

# Behavioral Modeling

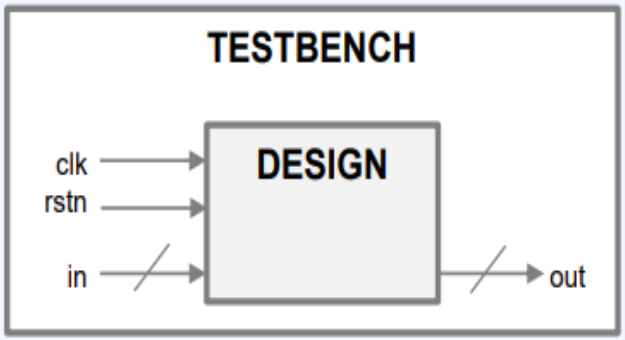
- Behavioral Modeling
  - Describe behavior in procedure blocks
  - This is usually in "*always*" block
  - This is normal way to implement sequential logic
    We can not save the value in variables with dataflow modeling (except tri-reg)
  - Let's deep-dive into behavioral modeling after practice




```
module mux_behavior(
    input   a, b,
    input   A, B, C, D,
    output  Q
);
    reg     Q;

    always @* begin
        case({b,a})
            'b00    : Q=A;
            'b01    : Q=B;
            'b10    : Q=C;
            'b11    : Q=D;
            default : Q=0;
        endcase
    end

endmodule
```

# Stimulus

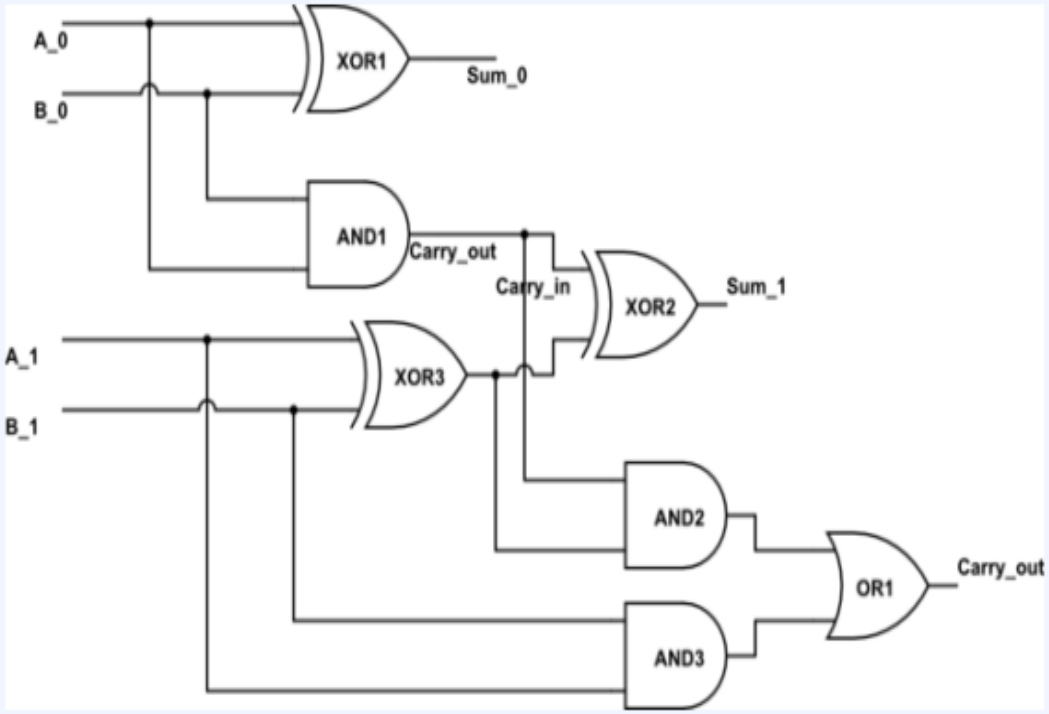
- Stimulus
  - We have only logic which is not triggered
  - If there is no value on input port, we can't see anything
  - to simulate, we need a stimulus logic called *testbench (we will focus on next chapter)*
  - *Let's try on EDA playground*




```
module mux_testbench;
    reg  [3:0]  a, b;
    reg  [3:0]  A, B, C, D;
    wire [3:0]  Q;

    mux_gatelevel u_mux_gatelevel(a[0], b[0], A[0], B[0], C[0], D[0], Q[0]);
    mux_dataflow  u_mux_dataflow (a[1], b[1], A[1], B[1], C[1], D[1], Q[1]);
    mux_behavior  u_mux_behavior (a[2], b[2], A[2], B[2], C[2], D[2], Q[2]);

    initial begin
        A='0; B='1; C='1; D='0;

        a='0; b='0;
        #10 a='1; b='0;
        #10 a='0; b='1;
        #10 a='1; b='1;
    end

    initial begin
        $monitor("gatelevel : a %d, b %d, Q %d",a[0],b[0],Q[0]);
        $monitor("dataflow  : a %d, b %d, Q %d",a[1],b[1],Q[1]);
        $monitor("behavior  : a %d, b %d, Q %d",a[2],b[2],Q[2]);
    end

endmodule
```

# MODELING PRACTICE

- Describe and simulate *2-bit full adder* in 3 modeling ways



# Behavior Modeling

- Procedure block
  - *initial* : execute only once at time zero
  - *always* : execute over and over, with condition defined at @

```
initial begin
    a = 0;
    a = ~a;
end

always @(posedge clk, negedge rstn)
if  (!rstn) a <= 0;
else        a <= ~a;
```

  - *begin-end* : sequential executes
  - *fork-join* : parallel executes

```
initial begin
    a=0; b=0; c=0;

    #10 a=1;    //time 10: a=1
    #20 b=1;    //time 30: b=1
    #30 c=1;    //time 60: c=1
end

initial begin
    a=0; b=0; c=0;
    fork
        #10 a=1;    //time 10: a=1
        #20 b=1;    //time 20: b=1
        #30 c=1;    //time 30: c=1
    join
end
```

  - net type can not be used in procedure block
  - Every procedure block executes concurrent.

# Behavior Modeling

- Blocking assignment
  - executes in order, sequential
  - previous line blocks next line.
    it is why they are called blocking
  - *a=b;*
- Non-Blocking assignment
  - executes in parallel
  - previous line doesn't block next line.
    it is why they are called non-blocking
  - *a<=b;*
  - *Do not mix blocking & non-blocking in a procedure*

```
initial begin
    #10 a=0;
    #20 a=1;
end

initial begin
    #10 b<=0;
    #20 b<=1;
end

initial begin
    c = #10 0;
    c = #20 1;
end

initial begin
    d <= #10 0;
    d <= #20 1;
end
```

```
initial begin
    a=0; b=1;
    #10
    a <= b;
    b <= a;
end

initial begin
    c=0; d=1;
    #11
    c = d;
    d = c;
end
```

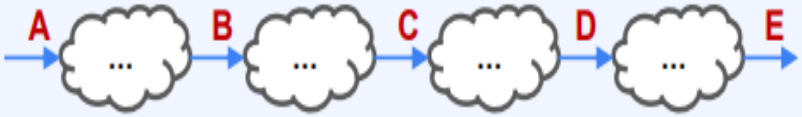
```
initial begin
    e=0; f=1;
    #12
    e <= f;
    f  = e;
end

initial begin
    g=0; h=1;
    #13
    g  = h;
    h <= g;
end

initial begin
    h=0; i=1;
    #14
    h <= i;
    #1
    i <= h;
end
```

# Behavior Modeling

- *always @(sensitivity list)*
  - always block executes when sensitivity list events
  - *always @(a, b)* means this procedure executes when a or b changes
  - *always @(*)* : * means any signal event who affects output of this procedure
  - *always @(posedge clk)* means this procedure excutes when clk is rising
  - *always @(negedge rstn)* means this procedure excutes when rstn is falling
  - *always @**, *always @(posedge clk, negedge rstn)* are most common usage

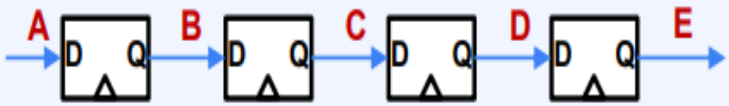
# Behavior Modeling

- ***always @****
  - This procedure executes when input signal of this block changes
  - ***It's like combinational logic !!***




  - ***Blocking assignment is highly recommended***
    *Because each signal should be followed by next one*

- ***always @(posedge clk, negedge rstn)***
  - This procedure executes when clock is rising (or rstn is falling)
  - ***It's like sequential logic (FF) !!***

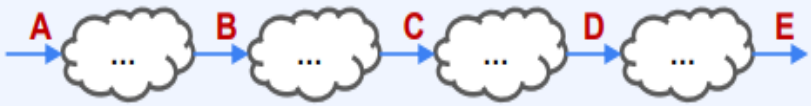



  - ***Non-blocking assignment is highly recommended***
    *Because "Q" should be affected by current "D", not follow previous signal*

# Behavior Modeling

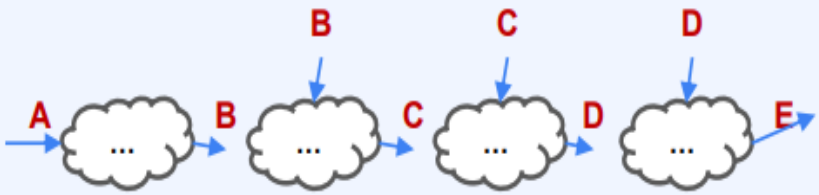
- *always @**
  - Non-blocking assignment makes unexpected results (Wrong implementation)
  - The connection can be cut off with non-blocking assignment
  - ***Avoid Non-blocking assignment in "always @(*)"***

**GOOD (blocking)**




```
initial #10 a = 1;

always @(*) begin
    b = a+1;
    c = b+1;
    d = c+1;
    e = d+1;
end
```

```
initial #10 a = 1;

always @(a) begin
    b = a+1;
    c = b+1;
    d = c+1;
    e = d+1;
end
```




**BAD (non-blocking)**

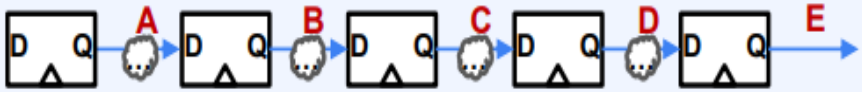
```
initial #10 a = 1;

always @(*) begin
    b <= a+1;
    c <= b+1;
    d <= c+1;
    e <= d+1;
end
```

```
initial #10 a = 1;

always @(a) begin
    b <= a+1;
    c <= b+1;
    d <= c+1;
    e <= d+1;
end
```

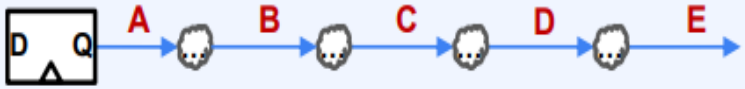
# Behavior Modeling

- *always @(posedge clk, negedge rstn)*
  - Blocking assignment makes that output is affected by the value being calculated, not the current state of input (what FF does)
  - Blocking assignment makes absence of Flip Flop (Wrong implementation)
  - ***Avoid Blocking assignment in "always @(posedge clk)"***

**GOOD (non-blocking)**




```
always @(posedge clk, negedge rstn)
if  (!rstn) begin
    {a,b,c,d,e} <= 0;
end else begin
    a <= 1;
    b <= a+1;
    c <= b+1;
    d <= c+1;
    e <= d+1;
end
```




**BAD (blocking)**

```
always @(posedge clk, negedge rstn)
if  (!rstn) begin
    {a,b,c,d,e} = 0;
end else begin
    a = 1;
    b = a+1;
    c = b+1;
    d = c+1;
    e = d+1;
end
```

# Conditional statement

- if-else
  - Role is as same as C language
  - begin-end grammar instead of { ... }
  - ```
    if (sel==0) begin
            Q=A;
    end else if (sel==1) begin
            Q=B;
    end else if (sel==2) begin
            Q=C;
    end else begin
            Q=D;
    end
    ```

- case
  - Role is as same as C language's switch-case
  - case-endcase grammar instead of switch{ ... }
  - ```
    case (sel)
            2'b00:      Q=A;
            //begin-end for over 2 lines
            2'b01:      Q=B;
            2'b10:      Q=C;
            default:    Q=D;
    endcase
    ```
  - when there are lots of selections, case statement is preferred to if-else statement
  - Case will be translated to *N:1 Mux* while if-else will be translated to N-1's *2:1 Mux*

# Looping statement

- ***for***
  - Same as C language
  - grammar : for (...) begin ... end
  - ```
    for (i=0;i<8;i++) begin
                    a[i] <= 0;
    end
    ```
  - *Only this Loop is normally used in Design (for reset initialization)*
- ***while***
  - Same as C language
  - grammar : while (...) begin ... end
  - ```
    while (ready == 0) begin
                    valid <= 1;
    end
    ```
- ***forever***
  - This loop never ends and executes continually
  - It should be used with timing construct
  - Normally, this is used for clock generation in TB
  - ```
    initial begin
                    clk = 0;
                    forever     #5 clk = ~clk;
    end
    ```
- ***repeat***
  - Executes fixed number of times
  - ```
    initial begin
                    x=0;
                    repeat (10)   #5 x++;
    end
    ```

# CLK, RST

- Design who implements Sequential Logic needs clock and reset from TestBench
- In this case, TestBench should have not only signal stimulus
  but also clock & reset generator

- Clock is usually generated by forever statements. Reset is usually generated in initial statements

```
initial begin
        clk = 0;
        forever #5 clk = ~clk;
end

initial begin
        rstn = 1;
        #10 rstn = 0;
        #20 rstn = 1;
end
```

# Common mistakes

- always @(posedge clk)
  - Avoid multiple driver

```
always @(posedge clk) begin
   if (a == 0) a <= 1;
   if (b == 0) a <= 0;
end
```

```
always @(posedge clk) begin
   if      (a == 0) a <= 1;
   else if (b == 0) a <= 0;
end
```

- always @*
  - See if all cases are allocated without exception
  - Define default value at the first line in always @* block

```
always @* begin
   case(a)
      0: b = 1;
   endcase
end
```

```
always @* begin
   b = 0;
   case(a)
      0: b = 1;
   endcase
end
```

- Combinational Loop
  - Combinational Loop can make simulator hang

```
assign a = a+1;

always @* b = b+1;
```

```
always @(posedge clk) a <= a+1;

always @(posedge clk) b <= b+1;
```